

Project Report on Pharmacy Management System

Aaditya Chaudary (Symbol. No. 312433)

Pratima Dahal(Symbol. No. 312488)

Yadav Pathak(Symbol. No. 312527)

Gomendra Multiple College

Affiliated to Purbanchal University

Bachelor of Computer Application

I Semester

Instructor: Jhalnath Chapagain

June, 2026



Affiliated to : Purbanchal University, Biratnagar (Nepal)
Faculty : Management (BBA), Science & Technology (BCA)
Under Gomendra Educational Foundation Pvt. Ltd.

श्री गोमेन्द्र बहुमुखी महाविद्यालय Shree Gomendra Multiple College

बिर्तामोड, (जहपा), नेपाल
Birtamode, (Jhapa) Nepal



023-540892
023-540437

Certificate

This is to certify that the project entitled “Pharmacy Management System”
submitted by Aaditya Chaudary (004-3-2-06823-2025), Pratima Dahal (004-3-2-06877-2025)
and Yadav Pathak (004-3-2-06916-2025), a student of Bachelor in Computer Application
(BCA), I semester, at Gomendra Multiple College has been approved as a bonafide work
carried out in partial fulfillment of the requirements for the degree of Bachelor in Computer
Application (BCA) under the affiliation of Purbanchal University.
This project has been examined and evaluated and is found to be satisfactory as per the
academic standards set by the institution.
We hereby recommend this proposal for further development and implementation as a part of
the final project work.

.....
Mr. Jhalnath Chapagain
Project Supervisor
Gomendra Multiple College, Birtamode

.....
Mr. Vimal Rajbanshi
Program Co-ordinator
Gomendra Multiple College, Birtamode

.....
Dr. Rupak Khanal
College Chief
Gomendra Multiple College, Birtamode

.....
Mr. External Examiner

Serving Educationally since 2053 BS / 1996 AD

email : gomendra405@gmail.com

website : gomendracollege.edu.np

Acknowledgement

I would like to express my sincere gratitude to all those who contributed to the successful completion of this project report on Pharmacy Management System. First and foremost, I am deeply thankful to the faculty members of Gomendra Multiple College, affiliated to Purbanchal University, for their continuous guidance and encouragement throughout this academic endeavor.

I am especially grateful to my project supervisor whose valuable suggestions, expert feedback, and unwavering support shaped the direction of this work. The knowledge imparted during the BCA program provided the strong foundation upon which this project was built.

I also extend my heartfelt thanks to my family and friends for their patience, moral support, and constant motivation. Their belief in my abilities kept me focused and determined during challenging phases of the project.

Finally, I acknowledge the contributions of the open-source C programming community whose resources and documentation proved invaluable in the development process.

Abstract

The management of Pharmacy records in Health institutions involves repetitive, error-prone manual processes that consume significant administrative time and resources. This project addresses that problem by designing and implementing a console-based Pharmacy Management System (SMS) using the C programming language. The system provides a structured, menu-driven interface through which administrators can perform core academic record-keeping operations including student enrollment, record retrieval, grade entry, and deletion of records.

The system utilizes file handling in C to persistently store student data, enabling records to survive program termination and remain accessible across multiple sessions. Key C programming constructs applied include structures (struct), arrays, loops, conditional logic, string manipulation functions, and file input/output using `fopen()`, `fread()`, `fwrite()`, and `fclose()`. The development environment consisted of GCC (GNU Compiler Collection) running on a Windows-based system.

Testing was conducted manually through structured test cases covering all CRUD (Create, Read, Update, Delete) operations. Results demonstrate that the system reliably adds, displays, searches, updates, and deletes student records. The project deepens understanding of file-based data persistence, procedural programming paradigms, and practical software development methodology. Future enhancements may include a graphical user interface (GUI), database integration using SQLite, and a web-based access portal.

Keywords: Pharmacy Management System, C programming, file handling, console application, CRUD operations.

Table of Contents

Certificate.....	i
Acknowledgement	ii
Abstract	iii
Table of Contents.....	iv
List of Tables	vi
List of Figures	vii
Chapter 1: Introduction	1
1.1 Background of the Project	1
1.2 Problem Statement.....	1
1.3 Objectives of the Project.....	2
1.4 Scope of the Project	2
1.5 Limitations	3
Chapter 2: Literature Review	4
2.1 Study of Existing System.....	4
2.2 Drawbacks of Existing System	5
2.3 Proposed System.....	5
2.4 Advantages of Proposed System.....	6
Chapter 3: System Requirements.....	7
3.1 Hardware Requirements.....	7
3.2 Software Requirements.....	7
3.3 Functional Requirements	7
Chapter 4: System Analysis	9
4.1 Feasibility Study	9
4.1.1 Technical Feasibility	9
4.1.2 Economic Feasibility	9
4.1.3 Operational Feasibility.....	9
4.2 System Workflow	10
Chapter 5: System Design.....	11
5.1 Data Structure Design	11
5.2 Menu Flow.....	11
5.3 Algorithm for Core Operations.....	12
5.3.1 Add Medicine.....	12
5.3.2 Search Medicine.....	12
5.3.3 Sell Medicine	12

5.3.4 Delete Medicine	13
Chapter 6: Implementation	14
6.1 Tools Used	14
6.2 Module Description	14
6.3 Key Code Snippets.....	15
6.3.1 Struct Definition and File Constant	15
6.3.2 Add medicine Function.....	15
6.3.3 Search medicine Function.....	17
6.3.4 Sell medicine Function	17
6.4 Sample Output	18
1 st . Sample Output.....	18
2 nd . Sample Output.....	19
3 rd . Sample Output	19
Chapter 7: Testing.....	20
7.1 Testing Strategy	20
7.2 Test Cases and Results.....	20
7.3 Bugs and Fixes.....	22
7.3.1. String Input Truncation.....	22
7.3.2. Infinite Loop on Invalid Inputs	22
7.3.3. Duplicate IDs	22
Chapter 8: Project Timeline	23
8.1 Project Scheduling using Gantt Chart.....	23
Chapter 9: Conclusion and Future Scope.....	24
9.1 Conclusion	24
9.2 Future Enhancements.....	24
Chapter 10: References	26

List of Tables

Table No.	Title	Page
Table 1	Hardware Requirement	7
Table 2	Software Requirement	7
Table 3	Tools Used	14
Table 4	Module Description	14
Table 5	Test Cases and Result	20

List of Figures

Figure No.	Title	Page No.
Figure 1	Gantt Chart	23

Chapter 1: Introduction

1.1 Background of the Project

In today's rapidly evolving healthcare environment, pharmacies play a vital role in ensuring the availability and proper distribution of medicines to patients. Managing medicine inventories, sales records, and stock levels manually can be time-consuming and prone to errors, particularly when dealing with a large variety of medicine products. As a result, there is a growing need for computerized systems that can streamline pharmacy operations and improve record management.

The Pharmacy Management System is a console-based application developed using the C programming language that aims to simplify and automate the management of pharmacy records. The system enables users to store, retrieve, update, and delete information related to medicines, including medicine ID, name, price, stock quantity, and expiry date. It also assists in tracking inventory levels and managing medicine sales efficiently.

1.2 Problem Statement

In pharmacies and medical stores, maintaining accurate records of medicines is essential for ensuring efficient inventory management and uninterrupted customer service. Traditionally many small pharmacies track medicine information, stock levels, and sales transactions manually. While these methods may be suitable for small-scale operations, they often become inefficient and difficult to manage as the number of medicines and transactions increases.

Manual record-keeping systems are prone to errors such as incorrect data entry, duplicate records, misplaced information, and inaccurate stock calculations. Tracking the availability of medicines, updating stock quantities and searching for specific medicine

details can be time-consuming tasks. The proposed system aims to address these challenges by providing a reliable, organized, and user-friendly solution for managing pharmacy data.

1.3 Objectives of the Project

The primary objectives of this project are as follows:

- To design and implement a console-based student management system using the C programming language.
- To apply file handling in C for storing medicine data permanently in external files.
- To implement CRUD operations - Create, Read, Update, and Delete - for medicine information.
- To demonstrate the application of C structures (struct) for organizing related data fields.
- To apply file I/O functions such as `fopen()`, `fwrite()`, `fread()`, and `fclose()` in a practical context.
- To maintain accurate records of medicines, including medicine ID, name, price, stock quantity, and expiry date.
- To practice modular programming by using user-defined functions for improved code organization.

1.4 Scope of the Project

The scope of this project is limited to the development of a console-based application for managing medicine records in small pharmacies, medical stores, and clinics. The system supports operations such as adding new medicine records, viewing available medicines, searching for specific medicines, updating existing information, deleting records and managing medicine sales. The system operates on a local machine as a single-user, offline system that stores data persistently using file handling techniques.

1.5 Limitations

Despite its utility, the system has several notable limitations. It does not include user authentication, meaning any user with access to the machine can modify records. The system lacks a graphical interface, which may present usability challenges for users unfamiliar with command-line environments. Concurrent access by multiple users is not supported. The system stores data in a flat text file without relational database structure, limiting the complexity of queries.

Chapter 2: Literature Review

2.1 Study of Existing System

Prior to the development of this system, medicine records at many small pharmacies were managed through one of three approaches: handwritten ledgers, Microsoft Excel spreadsheets, or commercially available pharmacy management software packages. Each of these approaches has been studied and assessed in the context of this project.

Handwritten registers represent the most traditional approach. Pharmacy staff record medicine names, IDs, stock levels, and expiry dates in physical notebooks. While simple to implement, this approach offers no search functionality beyond manual scanning, no backup mechanism, and no ability to efficiently track sales and automated stock deduction.

Spreadsheet-based management using tools such as Microsoft Excel offers improvement over paper registers in that data can be sorted, filtered, and computed. However, spreadsheets are not purpose-built for pharmacy management and require the user to manually implement validation rules and manage naming conventions across multiple sheets. They are also prone to accidental modification and do not enforce data integrity.

Commercial pharmacy management software, while comprehensive, are expensive, require internet connectivity, and are often too complex for small medical stores with limited IT infrastructure. Furthermore, their source code is not accessible, making customization impossible.

2.2 Drawbacks of Existing System

The existing approaches share a number of common drawbacks that justify the development of the proposed system:

- **Slow retrieval:** Searching for a specific medicine record in a paper register requires manual scanning, which is inefficient.
- **Data inconsistency:** Duplicate entries and typographical errors are common in manual systems with no validation.
- **No backup:** Paper records can be lost due to fire, water damage, or misplacement.
- **Stock inaccuracy:** Calculating accurate stock levels after numerous sales requires significant manual effort and is prone to errors.
- **Scalability issues:** As enrollment grows, manual systems become increasingly difficult to maintain.
- **Access control:** Physical registers offer no access restriction; any person with physical access can read or alter records.

2.3 Proposed System

The proposed Pharmacy Management System is a menu-driven, console-based application developed in C. It stores medicine records in a text file using C's file handling library. The system accepts input from the user through keyboard interaction and displays output on the console screen. Records are stored persistently so that data entered in one session remains available in the next.

The system organizes each medicine record using a C struct containing fields for ID (integer), name (character array), price (floating-point number), stock (integer), and expiry date. All operations are accessed through a numbered menu, making navigation straightforward for users without programming knowledge.

2.4 Advantages of Proposed System

- Fast record retrieval using ID-based search.
- Persistent data storage through binary file handling - records survive program termination.
- Reduced data entry errors through validation prompts.
- Simple console interface requiring no GUI framework or internet connection.
- Lightweight and portable - runs on any system with GCC installed.
- Cost-free - built using open-source tools with no licensing requirement.

Chapter 3: System Requirements

3.1 Hardware Requirements

The following hardware configuration represents the minimum requirement for running this application:

Table 1

Hardware Requirement

Hardware Component	Minimum Specification
Processor	Intel Pentium or equivalent, 1 GHz or higher
RAM	512 MB (1 GB recommended)
Storage	100 MB free disk space
Input Device	Standard keyboard
Display	VGA monitor or equivalent, 800x600 resolution

3.2 Software Requirements

Table 2

Software Requirement

Software Component	Specification
Operating System	Windows 7/10/11 or Ubuntu Linux 18.04+
Compiler	GCC (GNU Compiler Collection) 4.8 or higher
IDE / Editor	Code::Blocks, VS Code, or any text editor
Programming Language	C (C99 standard)

3.3 Functional Requirements

The system must support the following functional requirements:

- The system shall allow users to add new medicine records with ID, name, price, stock quantity, and expiry date.

- The system shall display all existing medicine records stored in the file.
- The system shall support search of a medicine record by ID.
- The system shall allow selling medicines and automatically deduct from stock.
- The system shall allow updating the details of an existing medicine record.
- The system shall allow deletion of a medicine record identified by ID.
- The system shall display a menu upon startup and return to the menu after each operation.
- The system shall save all records persistently in a text file.

Chapter 4: System Analysis

4.1 Feasibility Study

4.1.1 Technical Feasibility

The Pharmacy Management System is technically feasible. The C programming language is stable, widely supported, and available on all major operating systems. GCC, the compiler used for this project, is freely available and well-documented. Text file handling functions in C's standard library (stdio.h) are sufficient to implement all required data persistence operations. No additional libraries, frameworks, or external services are needed. The project is within the technical scope of a BCA first-semester student with knowledge of C fundamentals.

4.1.2 Economic Feasibility

The project incurs no financial cost. All tools used - GCC, Code::Blocks, and the C standard library - are freely available under open-source licenses. No server infrastructure, hosting fees, or licensed software is required. The development hardware required is a standard personal computer, which is already available in the college laboratory. The economic feasibility of this project is therefore highly favorable.

4.1.3 Operational Feasibility

The system is operationally feasible for small pharmacies and medical stores. Pharmacy staff familiar with basic computer usage can operate the console menu without specialized training. The menu-driven interface presents numbered options and clear prompts, minimizing the learning curve. The system does not require internet connectivity or network infrastructure, making it suitable for use in areas with limited technological resources.

4.2 System Workflow

The workflow of the Pharmacy Management System begins when the user launches the compiled executable. The program presents the main dashboard menu. The user selects an operation by entering the corresponding number. Based on the selection, the appropriate function is called. After the function completes, the program returns to the main menu. The user may exit the program by selecting option 7.

The high-level data flow is described as follows:

- **Input:** User provides medicine data (ID, name, price, stock, expiry date) or an ID for search/sell/update/delete.
- **Processing:** The program applies the corresponding logic (add record to file, read all records, locate record by ID, calculate sales, overwrite record, or remove record).
- **Output:** The program displays confirmation messages, retrieved records, generated bills, or error notifications on the console.
- **Storage:** All records are maintained persistently in text files in the working directory.

Chapter 5: System Design

5.1 Data Structure Design

The core data structure of the system is a C struct named Medicine. This struct serves the same conceptual role as a database schema, defining the fields and data types for each medicine record. The struct is defined as follows:

```
Struct Medicine {  
    int id;  
    char name[50];  
    float price;  
    int stock;  
    int expireMonth;  
    int expireYear;  
};
```

This structure helps organize the medicine data. The program uses standard file I/O operations such as `fprintf()` and `fscanf()` to handle data. The `id` field acts as the primary identifier for each record, analogous to a primary key in a relational database.

5.2 Menu Flow

The main menu presents the following six options to the user:

1. Add Medicine
2. View all available medicine
3. Search specific Medicine
4. Sell your Medicine
5. Update Medicine
6. Delete Medicine

7. Exit

Each menu option maps directly to a dedicated C function. The menu loop continues until the user selects option 7, at which point the program terminates gracefully.

5.3 Algorithm for Core Operations

5.3.1 *Add Medicine*

1. Open the text file in append mode.
2. Prompt user to enter ID, Name, Price, Stock, and Expiry Date.
3. Validate the input to ensure ID is unique and data is correct.
4. Write the record to the file using `fprintf()`.
5. Close the file. Display success message.

5.3.2 *Search Medicine*

1. Open the text file in read mode.
2. Prompt user to enter target medicine ID.
3. Read each record sequentially line by line.
4. Parse the line to extract the ID and compare it with the target value.
5. If match found, display record and return. If end of file reached, display 'Not found'.

5.3.3 *Sell Medicine*

1. Prompt user for medicine ID and quantity to sell.
2. Search for the medicine ID in the file.
3. Check if available stock is sufficient for the requested quantity.
4. If sufficient, generate a bill, calculate total cost, and update the remaining stock in the file.

5.3.4 Delete Medicine

1. Open the main text file in read mode and a temporary file in write mode.
2. Prompt user to enter the medicine ID to delete.
3. Read each record; write to the temporary file all records except the one to delete.
4. Close both files. Remove the original file and rename the temporary file to the original file name.

Chapter 6: Implementation

6.1 Tools Used

Table 3

Tools used

Tool	Purpose
GCC 9.4	Compiling the C source code
Code::Blocks 20.03	Integrated development environment (IDE)
Windows 10	Development and testing operating system
C99 Standard	Language specification used during compilation

6.2 Module Description

The system is organized into functional modules, each implemented as a separate C function. The `main()` function serves as the program entry point and contains the menu loop. The modules and their responsibilities are described below.

Table 4

Module Description

Function Name	Description
<code>create()</code>	Accepts input from the user and writes a new Medicine record to the file.
<code>viewAll()</code>	Reads and prints all medicine records from the file to the console.
<code>search()</code>	Searches for a medicine by ID and displays the matching record.
<code>sell()</code>	Manages medicine sales, calculates bill, and updates stock.

update()	Locates a record by ID and overwrites it with updated values via a temp file.
delete()	Removes a record identified by ID using the temp-file replacement technique.
Main()	Presents the menu loop, reads user choice, and calls the appropriate function.

6.3 Key Code Snippets

6.3.1 Struct Definition and File Constant

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

struct Medicine {

    int id;

    char name[50];

    float price;

    int stock;

    int expireMonth;

    int expireYear;

};
```

6.3.2 Add medicine Function

```
void create() {  
  
    //only important part of the code are snippet  
  
    struct Medicine med;  
  
    // Collect medicine details from the user  
  
    printf("Enter ID number: ");  
  
    scanf("%d", &med.id);  
  
    printf("Enter Medicine Name: ");  
  
    fgets(med.name, sizeof(med.name), stdin);  
  
    printf("Enter Medicine Price: ");  
  
    scanf("%f", &med.price);  
  
    // Save medicine record to file  
  
    fprintf(Pill, "%d|%s|%.2f|%d|%d/%d\n",  
  
        med.id,  
  
        med.name,  
  
        med.price,  
  
        med.stock,  
  
        med.expireMonth,  
  
        med.expireYear);  
}
```

6.3.3 Search medicine Function

```

void search() {

//only important part of the code are snippet

    // Read each record from the file

while (fgets(line, sizeof(line), Pill) != NULL) {

    // Extract the ID field

    char *token = strtok(line, "|");

    //Compare file ID with user input

    if (strcmp(token, search) == 0) {

        // Display the matching medicine record

        printf("Medicine Found!\n");

        break;

    }

}
}

```

6.3.4 Sell medicine Function

```

void Sell() {

// Search medicine by ID

if (strcmp(token, search_id) == 0) {

    // Check stock availability

    if (med.stock >= qty) {

        // Update stock quantity

        med.stock -= qty;

        // Calculate item total

        float item_total = med.price * qty;

        total_amt += item_total;
    }
}
}

```

```

        // Save updated record

        fprintf(Temp, "%d|%s|%.2f|%d|%d/%d\n",

                med.id, med.name, med.price,

                med.stock, med.expireMonth,

                med.expireYear);

        // Add item to bill

        fprintf(bill_file,

                "%-10d %-20s %-10d %-10.2f %-10.2f\n",

                med.id, med.name, qty,

                med.price, item_total);

    }

}

```

6.4 Sample Output

The following represents a sample console session demonstrating the add and display operations of the system.

1st. Sample Output

```

=====

Pharmacy Management

=====

1. Add Medicine

2. View all available medicine

3. Search specific Medicine

4. Sell your Medicine

```

5. Update Medicine
6. Delete Medicine
7. Quit
Enter your choice:1

2nd. Sample Output

```
=====
Pharmacy Management
=====
1. Enter ID number: 101
2. Enter Medicine Name: Vitamin C
3. Enter your Medicine price: 60
4. Enter Number of Stocks: 100
Enter Expire Date(M/Y): 2/2029
```

3rd. Sample Output

```
===YOUR INPUT DETAILS===
Id: 101
Medicine Name: Vitamin C
Price: Rs. 60.00
Stocks: 100
Expire Date: 2/2029
Press Enter to Continue
```

Chapter 7: Testing

7.1 Testing Strategy

Testing for this project was conducted exclusively through manual black-box testing. Each test case was designed to verify a specific functional requirement of the system. Test inputs were provided through the console interface, and outputs were observed and compared against expected results. Both valid inputs and edge-case inputs (such as non-existent roll numbers) were tested to assess system robustness.

7.2 Test Cases and Results

Table 5

Test Cases and Results

TC#	Description	Input	Expected Output	Actual Output	Status
TC01	Add a new medicine record	ID: 101, Medicine Name: Vitamin C Price: 60 Stock: 100 Expire Date: 2/2029	Record added successfully	Record added successfully	Pass
TC02	View all Medicine	Menu option 2	All stored records displayed in table format	All records displayed correctly	Pass

TC03	Search specific Medicine	ID: 101	Display data for 101	Data for ID 101 displayed	Pass
TC04	Sell Medicine	Bill ID: 101 How many Different Medicine to Sell: 1 ID: 101 Quantity: 20	Create a folder bill/*(bill_id).txt Calculate the total : Quantity*Price Display Bill of the medicine Update the stock	Display the Item sold and the total bill in the console.	Pass
TC05	Update existing record	What Do you Want to Update: 1.ID 2.Medicine Name 3.Medicine Price 4.Medicine Stock 5.Expire Date Enter your Choice: 2 Enter the ID: 101 Enter new Medicine Name: paracetamol	Successfully updated!	Successfully updated!	Pass

TC06	Delete existing Medicine	ID: 101	Record deleted successfully	Record deleted, no longer appears in display	Pass
------	--------------------------------	---------	--------------------------------	---	------

7.3 Bugs and Fixes

During testing, two notable issues were identified and resolved.

The Pharmacy Management System: Known Bugs & Resolutions

7.3.1. String Input Truncation

Issue: scanf() failed to read full medicine names containing spaces.

Fix: Replaced scanf() with fgets() to capture spaces, and used strcspn() to strip the trailing newline character.

7.3.2. Infinite Loop on Invalid Inputs

Issue: Entering a character when an integer was expected caused an infinite loop crash.

Fix: Added while(getchar() != '\n'); to clear leftover characters from the input buffer safely.

7.3.3. Duplicate IDs

Issue: Users could add multiple medicines with the exact same ID.

Fix: Added a validation loop using rewind() to check for duplicate IDs before allowing creation..

Chapter 8: Project Timeline

8.1 Project Scheduling using Gantt Chart

Figure 1

Gantt Chart

ID	Task Name	Week						
		Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7
1	Requirement Analysis							
2	Proposal Preparation							
3	System Design & Algorithm							
4	Coding & Development							
5	Testing & Debugging							
6	Documentation Writing							
7	Final Submission							

Chapter 9: Conclusion and Future Scope

9.1 Conclusion

This project successfully demonstrates the design and implementation of a console-based Pharmacy Management System using the C programming language. The system fulfills all stated objectives: it provides a menu-driven interface for adding, displaying, searching, selling, updating, and deleting medicine records, with persistent data storage achieved through file handling.

The development of this project reinforced several foundational concepts of the C language and software development in general. In particular, the application of C structures for organizing heterogeneous data, the use of file I/O functions from `stdio.h`, and the implementation of the temporary-file technique for record updating and deletion were valuable practical exercises. The project also introduced systematic testing methodology through manually designed test cases, improving the reliability and correctness of the final product.

The Pharmacy Management System, while simple in scope, demonstrates the practical utility of procedural programming for solving real-world administrative and inventory problems for small pharmacies. It provides a working foundation upon which more sophisticated features can be built in future iterations.

9.2 Future Enhancements

The following enhancements are proposed for future development of this system:

- Graphical User Interface (GUI): Replacing the console interface with a GUI developed using GTK or Qt would significantly improve usability and accessibility for non-technical users.

- **Database Integration:** Migrating from file storage to a relational database management system such as SQLite or MySQL would enable complex queries, enforce data integrity through primary keys and foreign keys, and support concurrent multi-user access.
- **User Authentication:** Implementing a login system with encrypted password verification would restrict unauthorized access and to view to other.
- **Automated Expiration Alerts:** Adding functionality to automatically track and alert users about medicines that are approaching their expiration dates.
- **Sales Analytics & Reporting:** Integrating advanced financial reporting features to automatically generate daily, weekly, or monthly revenue and inventory depletion reports.
- **Web Interface:** Developing a web-based front end using PHP or Python (Flask) connected to a database backend would enable access from any device with a browser, eliminating the need for local installation.

Chapter 10: References

Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.

King, K. N. (2008). C programming: A modern approach (2nd ed.). W. W. Norton & Company.

Balagurusamy, E. (2019). Programming in ANSI C (8th ed.). McGraw-Hill Education.

GNU Project. (2023). GCC, the GNU compiler collection. Free Software Foundation.
<https://gcc.gnu.org>

cppreference.com. (2024). C reference: Standard library. <https://en.cppreference.com/w/c>

TutorialsPoint. (2024). C programming tutorial.

<https://www.tutorialspoint.com/cprogramming/index.htm>

GeeksforGeeks. (2024). File handling in C. <https://www.geeksforgeeks.org/file-handling-c-classes>